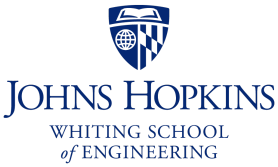
© Guven

Introduction to PyTorch



PyTorch on CUDA

Large neural networks can take advantage of the
GPGPU resources on your computer. Larger batch sizes that can fit in the GPU memory will be much more efficient on a GPU than a CPU.

This section provides an example PyTorch GPGPU application.

Windows 10 (x64)

PyTorch:

https://pytorch.org/
After a clean installation of `Anaconda3-2023.09-0-Windows-x86_64` run the following in Anaconda Prompt (Anaconda3),
`conda update -n base conda`
`conda update --all`

On the command line, use a separate Anaconda environment for PyTorch GPU:
`conda create -n torchgpu jupyter matplotlib pandas scikit-learn` ↩
`conda activate torchgpu` ↩
`conda install pytorch torchvision torchaudio pytorch-cuda=12.1 -c pytorch -c nvidia` ↩

Run this portion of the notebook under the virtual environment `torchgpu`.

Pre-requisites

1. The computer must have the CUDA toolkit **12.1** installed, the following link can provide installation instructions:

https://docs.nvidia.com/cuda/cuda-installation-guide-microsoft-windows/index.html

2. Download and Install cuDNN:

https://docs.nvidia.com/deeplearning/sdk/cudnn-install/index.html

3. Download `zlib123dllx64.zip` from (2.) and copy `zlibwapi.dll` to `C:\Windows\System32` and `C:\Windows\SysWOW64`.

In [1]:
```
import numpy as np

import torch

print(f'PyTorch version= {torch.__version__}')
print(f'CUDA available= {torch.cuda.is_available()}')  # Need CUDA and GPU
```

```
PyTorch version= 2.1.2
CUDA available= True
```

In [2]:
```
# CUDA Installation
print('CUDA Version')
!nvcc --version
print()

# CUDNN Installation
print(f'CUDNN Version: {torch.backends.cudnn.version()}')
print(f'Number of CUDA Devices: {torch.cuda.device_count()}')
print(f'Active CUDA Device: {torch.cuda.current_device()}')
print(f'Available devices: {torch.cuda.device_count()}, Name: {torch.cuda.
print(f'Current CUDA device: {torch.cuda.current_device()}')
```

```
CUDA Version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2022 NVIDIA Corporation
Built on Wed_Jun__8_16:59:34_Pacific_Daylight_Time_2022
Cuda compilation tools, release 11.7, V11.7.99
Build cuda_11.7.r11.7/compiler.31442593_0

CUDNN Version: 8801
Number of CUDA Devices: 1
Active CUDA Device: 0
Available devices: 1, Name: NVIDIA RTX 5000 Ada Generation
Current CUDA device: 0
```

In [3]:
```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer

df = pd.read_csv('/EP_datasets/stack_overflow_data.csv')
df = df[pd.notnull(df['tags'])]

X_tfidf = TfidfVectorizer(dtype=np.float32, max_features=1000).fit_transfo
y_tfidf = np.array(df.tags.astype('category').cat.codes, dtype='int64')
```

In [4]:
```
def kfold_eval_docs(_clf, _Xdocs, _ydocs):
    from sklearn.model_selection import StratifiedKFold
    from sklearn.metrics import accuracy_score

    # Need indexable data structure
    accuracy = []
    kf = StratifiedKFold(n_splits=10, shuffle=True, random_state=0)
    for train_index, test_index in kf.split(_Xdocs, _ydocs):
```

```
        _clf.fit(_Xdocs[train_index], _ydocs[train_index])
        y_pred = _clf.predict(_Xdocs[test_index])
        accuracy += [accuracy_score(_ydocs[test_index], y_pred)]
    return np.array(accuracy)
```

In [5]:
```
%%time
from sklearn.svm import LinearSVC

svc_acc = kfold_eval_docs(LinearSVC(dual=True, class_weight='balanced'), X

print(f"Linear SVM 10-fold CV accuracy= {np.mean(svc_acc):.2f} {chr(177)}{
```

```
Linear SVM 10-fold CV accuracy= 0.79 ±0.006
CPU times: total: 11.9 s
Wall time: 11.7 s
```

In [6]:
```
import numpy as np

import torch
import torch.nn as nn
import torch.optim as optim

# Set the GPU to device 0
gpu = torch.device('cuda:0')

class CustomMLPgpu(nn.Module):
    """ A PyTorch neural network model for GPGPU """
    def __init__(self, n_hidden=30, epochs=100, eta=0.05, minibatch_size=5
        super().__init__()
        self.n_hidden = n_hidden  # size of the hidden layer
        self.epochs = epochs  # number of iterations
        self.eta = eta  # learning rate
        self.minibatch_size = minibatch_size  # size of training batch - 1
        self.fc1, self.fc2, self.fc3 = None, None, None

    def _forward(self, X, apply_softmax=False):
        assert self.fc1 is not None

        # Carry the data to GPU
        X = nn.functional.relu(self.fc1(X.to(gpu)))
        X = nn.functional.relu(self.fc2(X))
        X = self.fc3(X)
        if apply_softmax:
            X = nn.functional.softmax(X, dim=1)
        return X

    def _reset(self):
        for m in self.modules():
            if isinstance(m, nn.Linear):
                m.reset_parameters()

    def predict(self, X):
        assert self.fc1 is not None

        # Carry the data to GPU
```

```
        net_out = self._forward(X.to(gpu, non_blocking=True), apply_softma
        p_values, indices = net_out.max(dim=1)

        # Carry the data back to CPU
        return indices.to('cpu')

    def fit(self, X_train, y_train):

        # Carry the data to GPU
        X_train = X_train.to(gpu, non_blocking=True)
        y_train = y_train.to(gpu, non_blocking=True).long()

        self._reset()  # Reset the neural network weights
        n_output= torch.unique(y_train).shape[0]
        n_features= X_train.shape[1]

        # Carry the input, hidden and output layers to GPU
        self.fc1 = nn.Linear(n_features, self.n_hidden).to(gpu)
        self.fc2 = nn.Linear(self.n_hidden, self.n_hidden).to(gpu)
        self.fc3 = nn.Linear(self.n_hidden, n_output).to(gpu)

        optimizer = optim.SGD(self.parameters(), lr=self.eta, momentum=0.9
        loss_func = nn.CrossEntropyLoss()

        for _ in range(self.epochs):
            indices = torch.arange(X_train.shape[0])

            for start_idx in range(0, indices.shape[0] - self.minibatch_si
                batch_idx = indices[start_idx:start_idx + self.minibatch_s
                optimizer.zero_grad()

                net_out = self._forward(X_train[batch_idx])

                loss = loss_func(net_out, y_train[batch_idx])
                loss.backward()
                optimizer.step()
```

In [7]:
```
%%time

acc = kfold_eval_docs(CustomMLPgpu(100, 100, 0.05, 1000).to(gpu),
                      torch.tensor(X_tfidf), torch.tensor(y_tfidf))

print(f"PyTorch Feedforward GPGPU NN 10-fold CV accuracy= {np.mean(acc):.2
```

```
PyTorch Feedforward GPGPU NN 10-fold CV accuracy= 0.78 ±0.007
CPU times: total: 23.7 s
Wall time: 22.2 s
```

In [8]:
```
%%time

acc = kfold_eval_docs(CustomMLPgpu(100, 200, 0.05, 2000).to(gpu),
                      torch.tensor(X_tfidf), torch.tensor(y_tfidf))

print(f"PyTorch Feedforward GPGPU NN 10-fold CV accuracy= {np.mean(acc):.2
```

```
PyTorch Feedforward GPGPU NN 10-fold CV accuracy= 0.78 ±0.007
CPU times: total: 24.7 s
Wall time: 22.4 s
```

In [9]:
```
%%time

acc = kfold_eval_docs(CustomMLPgpu(100, 200, 0.05, 5000).to(gpu),
                      torch.tensor(X_tfidf), torch.tensor(y_tfidf))

print(f"PyTorch Feedforward GPGPU NN 10-fold CV accuracy= {np.mean(acc):.2
```

```
PyTorch Feedforward GPGPU NN 10-fold CV accuracy= 0.78 ±0.007
CPU times: total: 12.2 s
Wall time: 10.1 s
```

In [10]:
```
%%time

acc = kfold_eval_docs(CustomMLPgpu(200, 200, 0.05, 2000).to(gpu),
                      torch.tensor(X_tfidf), torch.tensor(y_tfidf))

print(f"PyTorch Feedforward GPGPU NN 10-fold CV accuracy= {np.mean(acc):.2
```

```
PyTorch Feedforward GPGPU NN 10-fold CV accuracy= 0.78 ±0.006
CPU times: total: 26 s
Wall time: 24 s
```

In [11]:
```
%%time

acc = kfold_eval_docs(CustomMLPgpu(400, 200, 0.05, 2000).to(gpu),
                      torch.tensor(X_tfidf), torch.tensor(y_tfidf))

print(f"PyTorch Feedforward GPGPU NN 10-fold CV accuracy= {np.mean(acc):.2
```

```
PyTorch Feedforward GPGPU NN 10-fold CV accuracy= 0.78 ±0.008
CPU times: total: 28 s
Wall time: 26.6 s
```

Recall that the run time of the PyTorch feedforward neural network was around 4 minutes on a CPU.

In [1]:
```
%%html
<style>
    table {margin-left: 0 !important;}
    p {font-family: verdana;}
    li {font-family: verdana;}
    div {font-size: 10pt;}
</style>
<!-- Display markdown tables left oriented in this notebook. -->
```
